

# 第16课 执行计划进入 Worker

Scheduler 已经完成本轮决策：哪些请求执行、每个请求执行多少 Token、需要哪些 KV Blocks。 Scheduler.schedule() 将这些决定封装成 SchedulerOutput ，但它仍然只是EngineCore 进程中的 CPU 控制信息，不是模型能够直接使用的 GPU Tensor。

本课只回答一个问题：

SchedulerOutput 记录了什么，又怎样跨越 Executor 和进程边界，到达每个 GPU Worker？

下文以 vLLM v0.25.1 的 Model Runner V2 和本地多进程 MultiprocExecutor 为主。 UniProcExecutor 不需要进程间广播；Ray、外部 Launcher 和跨节点通信也不在本课展开。

## 一、 SchedulerOutput 是 Scheduler 与 Worker 之间的执行合同

Worker 要执行一轮模型计算，至少需要知道三件事：

LaTeX

▼

1

哪些请求需要在 Worker 中建立完整状态

2

+ 已驻留请求的状态发生了哪些变化

3

+ 每个请求本轮计算多少 Token

这三类信息分别由 scheduled\_new\_reqs 、 scheduled\_cached\_reqs 和 num\_scheduled\_tokens 承载。完整结构定义在 SchedulerOutput 。

| 字段                    | 传递内容                | Worker 侧用途               |
|-----------------------|---------------------|--------------------------|
| scheduled_new_reqs    | 请求的完整初始化快照          | 新建或重建 Worker 请求状态        |
| scheduled_cached_reqs | 已驻留请求的紧凑更新          | 更新进度、Token 和新增 Block IDs |
| num_scheduled_tokens  | req_id → 本轮 Token 数 | 决定本轮每个请求的 Query 长度       |
| finished_req_ids      | 已结束的请求 ID           | 删除 Worker 侧缓存状态          |
| preempted_req_ids     | 本轮被抢占的请求 ID         | 删除已经失效的 Worker 请求状态      |

Speculative Decoding、Encoder Input、KV Connector 和 Structured Output 还会增加扩展字段，但不会改变这份基本合同。

## 完整初始化： `NewRequestData`

Model Runner V2 中，两类请求通过 `NewRequestData` 进入 Worker：

- 第一次进入 Worker 的新请求；
- 被抢占后重新调度、需要重建 Worker 状态的恢复请求。

`NewRequestData` 携带 `req_id`、Token IDs、Sampling/Pooling 参数、完整 Block IDs、`num_computed_tokens`、LoRA 和多模态信息。定义见 `NewRequestData`。

这里的 `num_computed_tokens` 不一定是 0。新请求可能命中 Prefix Cache，恢复请求也可能重新获得有效前缀；Worker 必须从这个进度继续，而不是无条件从 position 0 开始。

V2 路径会先把 `scheduled_resumed_reqs` 并入 `scheduled_new_reqs`，再通过 `NewRequestData.from_request()` 构造完整快照：`Scheduler.schedule()`。

## 紧凑更新： `CachedRequestData`

持续运行且仍驻留 Worker 的请求不需要每轮重发完整 Prompt 和采样参数，只需发送：

- 本轮参与执行的请求 ID；
- 当前 `num_computed_tokens` 和 `num_output_tokens` 快照；
- 新分配的 Block IDs；
- 部分 Pipeline Parallel 路径需要补充的 Token IDs。

结构定义见 `CachedRequestData`，数据由 `Scheduler._make_cached_request_data()` 从 Scheduler 侧 `Request` 生成。

“紧凑更新”不等于每个字段都是增量：

```
1  new_block_ids
2  → 本轮新分配的 Block IDs，只需 append
3
4  num_computed_tokens / num_output_tokens
5  → 构造计划时的当前计数快照
```

恢复请求在 V2 中不走 cached-request 的 append 路径，而是通过 `NewRequestData` 重建完整 Worker 状态，因此不会把新 Block IDs 追加到抢占前已经失效的映射上。

## 二、一份计划怎样描述本轮执行区间

考虑一个只有一个 KV Cache Group、`block_size = 16` 的批次：下文为便于阅读，省略单个 KV Cache Group 的外层 tuple。例如源码中的 `([5],)` 简写为 `[5]`。

| 请求 | Worker 状态 | 本轮起点 | 本轮 Token 数 | 工作类型               |
|----|-----------|------|------------|--------------------|
| R1 | cached    | 31   | 1          | decode-like        |
| R2 | cached    | 8    | 16         | 继续执行 Prefill Chunk |
| W1 | new       | 0    | 15         | 新请求 Prefill        |

对应的计划可以压缩成：

Python

```
1 scheduled_new_reqs = [W1]
2 scheduled_cached_reqs.req_ids = [R1, R2]
3 num_scheduled_tokens = {"R1": 1, "R2": 16, "W1": 15}
4 total_num_scheduled_tokens = 32
```

再假设本轮相关的 Block 信息为：

LaTeX

```
1 ▾ R1: Worker 已有 [37, 12], 本轮没有新增 Block
2 ▾ R2: Worker 已有 [81], 本轮追加 Block 44
3 ▾ W1: 完整 BlockTable 为 [5]
```

因此发送给 Worker 的不是三份相同形态的数据：

LaTeX

```
1 R1 → CachedRequestData(new_block_ids=None)
2 ▾ R2 → CachedRequestData(new_block_ids=[44])
3 ▾ W1 → NewRequestData(block_ids=[5])
```

计划进入 Worker 后，R2 的映射会从 `[81]` 变为 `[81, 44]`。Block IDs 由 Scheduler 决定；Worker 这里只接收并物化这项决定，不重新分配 Block。

### 执行计划必须同时记录“起点”和“长度”

`num_scheduled_tokens` 只回答“计算多少个 Token”。Worker 还需要知道“从哪个 position 开始”。对每个请求，本轮区间都是：

```
1 start = num_computed_tokens
2 count = num_scheduled_tokens
3
4 positions = [start, start + count)
```

三个请求对应：

| 请求 | start | count | 本轮 positions | 本轮结束位置 |
|----|-------|-------|--------------|--------|
| R1 | 31    | 1     | 31           | 32     |
| R2 | 8     | 16    | 8—23         | 24     |
| W1 | 0     | 15    | 0—14         | 15     |

以 R2 为例，Scheduler 必须先冻结 `start=8`，Worker 才能构造 positions 8—23。仅发送 `count=16`，Worker 无法判断这 16 个 Token 对应 positions 0—15、8—23，还是其他区间。

## Scheduler 为什么在返回计划前就推进进度

`Scheduler.schedule()` 的顺序是：

```
1 R2 Request.num_computed_tokens = 8
2   |
3   |— 构造快照: start=8, count=16
4   |— 构造 SchedulerOutput
5   |— _update_after_schedule(): 8 → 24
6
7 return SchedulerOutput(start=8, count=16)
8   |
9   |— Worker 执行 positions 8-23
```

也就是说，计划保留本轮执行起点，而 Scheduler 内部的 `Request` 提前站到预计完成位置。源码先构造 `SchedulerOutput`，随后调用 `_update_after_schedule()`：

- `SchedulerOutput` 的构造与返回顺序
- `Scheduler._update_after_schedule()` 推进请求进度

这种推进是 Scheduler 的乐观记账：positions 8—23 已经被安排给当前 step，后续调度不应重复选择它们；它不是“GPU 已经同步执行完毕”的证明。

正常执行时，`update_from_output()` 不会再次增加这 16 个 Token。只有实际有效工作量小于计划值时才需要纠偏。例如 Speculative Decoding 拒绝部分 Draft Tokens，Scheduler 会从 `Request.num_computed_tokens` 中减去 `num_rejected`：`Scheduler.update_from_output()`。

### 三、EngineCore 把计划交给 Executor

`EngineCore.step()` 将调度、执行和结果更新串成一轮：对应源码是 `EngineCore.step()`。

```
Python |
1 scheduler_output = self.scheduler.schedule(...)
2 future = self.model_executor.execute_model(scheduler_output, non_block=True)
3 grammar_output = self.scheduler.get_grammar_bitmask(scheduler_output)
4 model_output = future.result()
5 if model_output is None:
6     model_output = self.model_executor.sample_tokens(grammar_output)
7 self._process_aborts_queue()
8 engine_outputs = self.scheduler.update_from_output(
9     scheduler_output, model_output
10 )
```

`non_block=True` 使模型执行先返回一个 `Future`。EngineCore 可以在 Worker 执行期间准备 Structured Output 的 Grammar Bitmask，随后在 `future.result()` 等待模型结果。如果执行阶段没有直接产生最终采样结果，EngineCore 再把 Grammar Bitmask 交给 `sample_tokens()`；处理本轮执行期间到达的 abort 后，才调用 `update_from_output()`。这改变的是 CPU 控制流的重叠方式，不会把 `SchedulerOutput` 变成异步可变对象。

从 `EngineCore` 到 Model Runner V2 的调用层级如下：

- `EngineCore.step()`
  - `Scheduler.schedule()`
    - 构造 `SchedulerOutput`
  - `model_executor.execute_model(scheduler_output, non_block=True)`
    - `MultiprocExecutor.execute_model()`
      - `collective_rpc("execute_model", ...)`
        - `rpc_broadcast_mq.enqueue(...)`
          - 每个 Worker 进程运行 `WorkerProc.worker_busy_loop()`

- `rpc_broadcast_mq.dequeue(...)`
- `getattr(self.worker, method)`
- `gpu_worker.Worker.execute_model(scheduler_output)`
  - `gpu.model_runner.GPUModelRunner.execute_model(...)`

调用链在这里先停住。 `GPUModelRunner.execute_model()` 怎样把请求快照写入持久状态、怎样组织

当前批次以及怎样生成 GPU 输入，属于 Worker 内部的下一段数据变换。

## 四、MultiprocExecutor 广播执行，只收集一个普通结果

`MultiprocExecutor.execute_model()` 本身不执行模型，而是发起一次 collective RPC：

```

1  self.collective_rpc(
2      "execute_model",
3      args=(scheduler_output,),
4      unique_reply_rank=self.output_rank,
5      non_block=non_block,
6  )

```

这里存在两个方向不同的动作：

```

1  执行扇出
2  → 每个 Worker rank 收到等价的 execute_model(scheduler_output) 调用
3
4  普通模型结果汇入（未启用 KV 输出聚合器）
5  → Executor 只从 output_rank 的 response queue 取得普通 ModelRunnerOutput

```

`unique_reply_rank` 只限制普通返回值从哪个 rank 收集，不表示只有 `output_rank` 执行模型。

消息入队与 response queue 的选择位于 `MultiprocExecutor.collective_rpc()`。

启用 KV Connector 的输出聚合器后，Executor 会读取需要参与聚合的 rank 返回值，再由 `kv_output_aggregator` 合并；这不会改变执行请求仍广播给所有 Worker 的事实。

广播队列传递的是等价的控制负载，不是跨进程共享的同一个 Python 对象，也不是预先整理好的 GPU `input_ids`、`positions` 或 Attention Metadata。每个 Worker 都要在自己的进程和设备上消费

这份计划。

## 同一份计划不等于相同的本地计算

所有 rank 对请求顺序、本轮 Token 数和 Block IDs 必须有一致理解，但各 rank 的模型参数和KV Tensor 属于本地设备：

| 并行方式 | 各 rank 接收的控制计划      | 各 rank 的实际工作                                            |
|------|---------------------|---------------------------------------------------------|
| TP   | 等价的 SchedulerOutput | 计算本 rank 的权重、Attention Head 或 KV Head 分片，并参与 Collective |
| PP   | 等价的 SchedulerOutput | 只执行本 Stage 的层；非首 Stage 接收中间激活，非末 Stage 发送中间激活           |

TP rank 可以使用相同 Block IDs 索引各自 rank-local 的 KV Cache Tensor；PP rank 则只保存本 Stage 所属 Attention Layers 的 KV。相同 Block ID 只保证各 rank 对逻辑页列的编号一致，不代表多张 GPU 共享同一块物理显存。

PP 的激活收发发生在 `gpu_worker.Worker.execute_model()`：非首 Stage 在 forward 前接收 `IntermediateTensors`，非末 Stage 在 forward 后发送它们。

## Worker 入口处的交接边界

计划到达 Model Runner V2 后，入口顺序是：

LaTeX

```
1 GPUModelRunner.execute_model(scheduler_output)
2   |─ finish_requests()           删除已完成或被抢占的请求状态
3   |─ free_states()              释放已失效的 Encoder Cache 状态
4   |─ add_requests()             用完整快照建立新状态
5   |─ update_requests()          应用 cached-request 更新
6   |─ block_tables.apply_staged_writes()
7   └─ prepare_inputs()           开始构造本轮 GPU 输入
```

源码入口见 `GPUModelRunner.execute_model()`。

本课关注的交接在 `execute_model(scheduler_output)` 完成：Scheduler 的控制计划已经跨进程到达每个 Worker。`RequestState`、`req_state_idx`、`InputBatch`、`BlockTables` 和 `Slot Mapping`解释的是计划到达之后怎样变成 GPU 可消费的数据，不再属于跨进程传递本身。

## 关键结论

1. `SchedulerOutput` 是 Scheduler 与 Worker 之间的 CPU 控制合同，不是 GPU 输入 Tensor。
2. `NewRequestData` 提供完整初始化快照；`CachedRequestData` 提供已驻留请求的紧凑更新。
3. 一份计划必须同时携带本轮起点 `num_computed_tokens` 和本轮长度 `num_scheduled_tokens`。
4. Scheduler 先冻结起点，再乐观推进自己的 `Request`；必要时根据实际输出回退。
5. `MultiprocExecutor` 把等价的调用广播到所有 Worker，`output_rank` 只决定普通结果从哪里返回。
6. 相同 Block IDs 在不同 rank 上索引各自的本地 KV Tensor，不表示共享同一块 GPU 显存。